

skycell: a cost-based HEALPix index for sky positions in PostgreSQL

Design, brute-force validation, and a controlled comparison with Q3C and pgSphere

J. Salgado¹  and M. Delli Veneri¹ 

SKA Observatory, Jodrell Bank, Lower Withington, Macclesfield SK11 9FT, United Kingdom
jesus.salgado@skao.int, michele.delliveneri@skao.int

Received ...; accepted ...

ABSTRACT

Context. Astronomical archives answer positional queries from relational databases, and in the PostgreSQL world two extensions dominate: Q3C, which keys each source by an integer drawn from a quad-tree on a cube and stores it in a B-tree, and pgSphere, which stores spherical geometry in a GiST tree. The first is compact and fast for points; the second is general and indexes regions as well.

Aims. We ask whether the two can be combined: a single index that keeps the integer key and the B-tree, yet answers cones, polygons, cross-matches and stored footprints in archives’ own geometry language, establishing where it helps, where it does not, and what its cost model depends on.

Methods. We present **skycell**, a PostgreSQL extension that keys positions by their order-29 HEALPix cell and turns a query region into index ranges. How finely to cut is decided by a cost model – the price of an extra index range against the rows it removes – with local source density read from the histogram **ANALYZE** already keeps. Cells are classified conservatively against the region, and a planner support function rewrites the predicate into B-tree range conditions plus an exact test, or, for a constant cone, a custom scan walks the covering’s ranges itself; a companion GiST opclass family serves stored regions directly. Coverings are validated by brute force against millions of catalogue points, including adversarial cones and polygons. Performance is measured against Q3C 2.0.5 and pgSphere 1.5.2 on 10- and 50-million-row corpora – synthetic and resampled from the real Gaia DR3 density field – in trials where all three methods answer the same query back to back in randomized order, analysed paired, warm and cold.

Results. Warm, skycell’s custom scan beats pgSphere by 17–33% at every radius from 1'' to 3° on both synthetic corpora, where the range rewrite alone is within 12% of it below 30'; cold it is faster or level at every radius (ratios 0.51–0.99). Its index is 214 MB against pgSphere’s 685 MB – 38 GiB against 160 GiB at Gaia DR3 scale for the pgSphere-plus-Q3C pair covering the same ground – builds 30 times faster, answers convex polygons 2.7–4.4 times faster, cross-matches 25–33% faster than **q3c_join** and 3.2–3.8 times faster than pgSphere through its custom scan (on the resampled corpus; on real positions, through the range rewrite, it is 1.32 of **q3c_join**), and its row estimates are wrong by a factor 1.2 rather than 2.5–3.0. Its one free cost parameter is derived from the relation’s own statistics, its chosen cell order costs about 3% against the best available one, and its density estimate is good to within a factor 1.6 on ordinary sky (low by 10–100× in a globular cluster core, which costs the covering chosen at most a factor 1.8). Raising the catalogue to 50 million rows widens the advantage for regions of 30' and larger while cones of an arcminute and below do not move. A **float4** GiST box key matched to pgSphere’s own representation beats every region-opclass alternative tried, including pgSphere’s native GiST, on every corpus measured. An attempt to sharpen the covering’s own search further looked like a clean win in execution time alone but reversed under honest total-time accounting – reported as a methodological caution, not shipped as a default.

Conclusions. A cost-based covering over an equal-area integer key is a sound basis for a sky index, and the right question is not whether it is faster but for which queries. Its cost is paid per query and its benefit per row scanned, so it wins where enough rows are scanned to amortise a covering – degree-scale regions, polygons, cross-matching, cold caches – and that boundary moves up in radius as the catalogue grows denser. On a Gaia-sized catalogue a small cone search is not the workload this index improves.

Key words. Astronomical databases: miscellaneous – Catalogs – Virtual observatory tools – Methods: data analysis

1 Introduction

Almost every question asked of an astronomical archive is positional: what is within this cone, which observations cover this field, which of my sources match that catalogue. In the Virtual Observatory these arrive as ADQL (Ortiz et al. 2008) through the Table Access Protocol (Dowler et al. 2019), and the service must answer them with an index.

Two PostgreSQL extensions carry most of this traffic. Q3C (Koposov & Bartunov 2006) projects the sphere onto a cube, subdivides each face with a quad-tree, and numbers the cells along a space-filling curve, so each source is one 64-bit integer in a B-tree. pgSphere (Chilingarian et al. 2004) adds spherical types indexed with GiST over bounding volumes, so regions can be stored and queried as data. The scale this has to work at is set by archives already in op-

eration: the ESA Gaia Archive (Salgado et al. 2017, 2020) serves order 10^9 sources and lists cross-match among its server-side capabilities (Sect. 6.5). Neither idea is new to astronomy – the Hierarchical Triangular Mesh (Kunszt et al. 2001) already keyed positions by a subdivided polyhedron, and the zones algorithm (Gray et al. 2006) pushed a purely relational arrangement for cross-matching – but catalogue size has grown and most queries now arrive through a generated layer (e.g. Demleitner et al. 2014) rather than being hand-written.

The two designs trade off: a linear key in a B-tree is compact and cheap to probe, but a region must become a set of key ranges first, and that approximation’s quality depends on how finely the region is cut; a tree of bounding volumes needs no such step but has larger, overlapping keys and imposes no order on the table. This paper presents `skycell`, which keeps the integer key and asks a cost question at the point where the classical approach fixes a constant: how finely should a region be cut? The answer depends on two quantities the database already knows – local source density and the price of an extra index range – and is computed rather than configured.

The measurements are built to survive scrutiny rather than flatter the method: every query is answered by all three methods back to back in randomized order, analysed paired; everything is measured warm and cold; and one corpus is density-matched to the real Gaia DR3 sky while another is real, unresampled Gaia DR3 positions, not a density field standing in for them. Sect. 2 develops the B-tree index, Sect. 3 the GiST alternative for stored regions, Sect. 4 what is proven versus tested, Sect. 5 the ADQL surface, Sect. 6 the benchmark and results (source-like catalogues, cross-matches, stored regions), and Sect. 7 what this does and does not establish.

2 The B-tree index

2.1 An equal-area integer key

A position is keyed by its order-29 HEALPix cell in the nested scheme (Górski et al. 2005) ($12 \cdot 4^{29} \approx 3.5 \times 10^{18}$ cells of about 0.4 mas), computed by the standard `ang2pix_nest` arithmetic and stored as a signed 64-bit integer – a fixed-order index, not a multi-order key, so a B-tree orders keys along the HEALPix curve. Three properties follow, only the first shared with Q3C’s cube quad-tree. Cells are *nested*: the order-29 descendants of a cell p at order k are exactly the contiguous interval

$$[p \cdot 4^{29-k}, (p+1) \cdot 4^{29-k} - 1], \quad (1)$$

so a region covered by cells of any orders becomes a union of integer intervals a B-tree answers with range scans (the MOC standard’s NUNIQ encoding (Fernique et al. 2014), used for stored regions, Sect. 3,

packs the order into the key as $4 \cdot 4^k + p$, related by Eq. (1)). Cells are *equal area*, so keys-per-interval is proportional to source density per solid angle: the ANALYZE histogram is therefore a crowding map (Sect. 2.2) with no extra machinery. And cells are *standard* – the same numbering underlies Gaia’s `source_id` (Gaia Collaboration 2016) and IVOA MOC maps (Fernique et al. 2014) – so keys, coverings and footprints are interoperable rather than private to one extension.

2.2 Coverings decided by cost

A query region is covered by cells whose union contains it; the covering becomes range conditions on the key, and rows inside the covering but outside the region are removed by an exact test. Cutting finer means fewer false positives and more ranges. With cells of size s covering a region of radius r and local source density ρ ,

$$C(s) = n_{\text{ranges}}(s) c_{\text{range}} + \rho (A_{\text{cover}}(s) - A_{\text{region}}), \quad (2)$$

both terms in rows: the second is the expected false positives, and c_{range} converts one additional index range into the false-positive rows it is worth avoiding – a property of the installation (Sect. 6.4 measures it). A covering follows the region’s boundary, so for $s \ll r$ the ranges grow as r/s while the wasted area grows as the perimeter times s ; writing $A_{\text{cover}} \simeq \pi(r + \beta s)^2$ and $n_{\text{ranges}} \simeq \gamma r/s$ and minimising Eq. (2) gives

$$s_{\star} = \sqrt{\frac{\gamma c_{\text{range}}}{2\pi\beta\rho}}, \quad (3)$$

independent of region size: sky density and range price alone decide how finely to cut. For $s \gtrsim r$ the balance differs, so rather than solve the branches the implementation evaluates Eq. (2) at all 30 orders and takes the cheapest, letting a range cap and a cell-size guard simply remove candidates. $\beta = 0.7$ and $\gamma = 2$ are geometric; Sect. 6.4 shows the order choice is flat enough near its optimum not to need them exact.

The same 0.25° cone covered in two places on the same catalogue illustrates both regimes: at high Galactic latitude, 11 sources leave almost nothing to exclude, so the model buys one range of large cells (5.6 times the cone’s area); in the bulge, 1183 sources make the false positives cost more than extra ranges save, so the model cuts two orders deeper (23 cells, 8 ranges, 1.3 times the area).

Two guards keep the model safe against its own inputs: a partly covered cell is never allowed to exceed a multiple (64 by default) of the region’s own area, bounding the damage from a dense cluster the histogram underestimates; and gaps between ranges are merged when the rows they contain cost less than the range they save.

The density estimate. PostgreSQL’s `ANALYZE` stores, for the indexed expression, an equi-depth histogram: a sorted list of $B+1$ bounds $h_0 < h_1 < \dots < h_B$ in key space such that each bucket $[h_i, h_{i+1})$ holds about N/B of the table’s N rows. Because the key is an equal-area cell index, the rows in a key interval $[a, b]$ are estimated by linear interpolation within the buckets it spans,

$$\hat{n}(a, b) = \frac{N}{B} \sum_i \frac{|[a, b] \cap [h_i, h_{i+1})|}{h_{i+1} - h_i}, \quad (4)$$

and the corresponding density in rows per steradian is

$$\hat{\rho}(a, b) = \frac{\hat{n}(a, b)}{(b - a + 1) A_{29}}. \quad (5)$$

The estimate used by Eq. (3) is $\hat{\rho}$ evaluated over the order- m cell containing the region’s centre, with m chosen so that $s_m \approx 4r$: large enough to contain enough rows to be a density, small enough to be local.

Three caveats: the quantity is a density, not the planner’s own selectivity (computed separately from the same histogram); Eq. (4) assumes rows are spread uniformly inside a bucket, which fails for a cluster smaller than one, hence the area guard of Sect. 2.2; and the estimate is only as local as a bucket (with $B = 1000$, $N = 10^7$, a bucket holds 10^4 rows). Section 6.4 measures how close $\hat{\rho}$ lands to the truth.

Classifying cells: what is bounded and what is not. A cell must be classified against the region as inside, outside or straddling its boundary, and the three outcomes are not symmetric: a cell called *outside* is dropped, so if that is ever wrong the query silently loses rows, while *inside* and *straddling* cells both produce a superset the exact predicate (Sect. 2.3) trims back. Only the outside decision therefore needs a bound rather than an estimate.

The bound used is

$$R_{\text{cell}}(k, p) = \min(\text{maxpixrad}(k), \max_i \angle(\mathbf{c}_{k,p}, \mathbf{v}_i)), \quad (6)$$

where $\mathbf{c}_{k,p}$ is the cell’s centre, $\mathbf{v}_i$ its corners, and the first term the analytic worst case the reference HEALPix implementation uses over the whole sphere. A cell is outside when $\angle(\mathbf{q}, \mathbf{c}_{k,p}) - R_{\text{cell}} > r + \epsilon$ for a cone of centre $\mathbf{q}$, radius r (correspondingly per bounding half-space for a polygon); $\epsilon = 2 \times 10^{-13}$ rad covers accumulated floating-point error, five orders of magnitude below the leaf cell’s resolution. The inside test is tighter, using distances to the cell’s own corners and edge midpoints directly. A tempting outside-test alternative (distance to the chords between corners) is not a bound – it can under-estimate by a factor reaching 159 near order 18, invisible to every test we had until it wasn’t; Eq. (6) avoids it, at a cost of 2.5% more ranges and no measurable time.

2.3 Planner integration and ingestion

The extension is written in C against the PostgreSQL 15+ server API: a geometry layer with no database dependencies and a layer connecting it to the planner via a *planner support function* (`SupportRequestSimplify`, the mechanism PostGIS uses for `STDWithin`). Before path generation, a predicate such as `point <@ circle(...)` – any region, unless it is a constant cone, which has a scan of its own (below) – is replaced by a disjunction of range conditions on the key (`cell BETWEEN lo1 AND hi1 OR ...`) conjoined with an exact test, kept only if the relation carries an index on the position. The index conditions are ordinary range predicates, so the planner estimates their selectivity from the same histogram, and the exact test’s own argument carries the covering’s occupied fraction, keeping the product of the two estimates close to the truth. Two caches matter for planning cost: the density model and index identity per backend, and coverings memoised by region and statistics source (worth 45–70 and up to 50 μs per plan on a TAP corpus). For joins, where the region is not constant, the rewrite emits a fixed number of range slots per outer row, as `q3c_join` does; the custom scan below covers each outer row’s cone instead, and Sect. 6.5 shows the difference matters.

A custom scan for constant cones. The rewrite leaves the ranges to PostgreSQL, which builds an index path for every arm of the disjunction, estimates each against the histogram and costs a `BitmapOr` over them: about 10 μs per range on the catalogues here, more than computing the covering, and enough to decide a small cone warm (Sect. 6.4). A cone search by constant centre and radius on one relation, the common case, is therefore answered by a planner custom scan instead (`skycell.custom_scan`, the default; heap tables only). The support function declines to rewrite such a cone and answers the planner’s selectivity request for it instead, with the rows the covering expects scaled by the cone’s share of the covering’s area, the same product the rewrite’s two estimates make; the planner sees one clause where it saw one per range. A path hook then offers one custom path wherever a B-tree on the cell expression can serve it, costed from the covering as PostgreSQL costs the same work (a descent per range, the index entries, the heap, the recheck). The node walks the covering’s ranges in one of two modes, chosen at plan time the way PostgreSQL chooses between an index and a bitmap scan, from the column’s correlation statistic: *ordered*, one index scan per range in cell order, when the heap follows the cell order; *bitmap*, every range’s TIDs collected into one bitmap and each heap page read once in physical order, when it does not – an ObsCore table clustered by collection, say. Rows are rechecked with the exact test, which

needs no cell computed per row. Because no range is planned, a range is priced as the scan pays for it rather than as the rewrite does (Eq. 8), and coverings come out finer. Only a cone that is a top-level AND term of a WHERE or JOIN ON is kept for the scan, since only there can it restrict one; under an OR it is rewritten as before, so a `BitmapOr` over its ranges and the other arm’s index still serves it. A cross-match cone, whose centre or radius comes from another relation’s row, gets the same node parameterized by that row, on the inner side of a nested loop: each outer row’s cone is covered when the node is rescanned for it, with as many ranges as that cone needs rather than the rewrite’s fixed slots; a generic plan or a correlated sub-select covers its cone per rescan likewise, and so do the Q3C-shaped spellings (`skycell_join`, `skycell_radial_query`) through the cell expression they synthesise. Every non-cone region keeps the rewrite; before PostgreSQL 18 the bitmap mode does not prefetch heap pages. A regression suite checks that every query it holds returns the same rows with the custom scan on and off, in both modes, rescanned in a nested loop, under a generic plan and through a scrollable cursor, on PostgreSQL 16 and 18.

Ingestion and maintenance. The index is an ordinary B-tree on an expression: one 8-byte key per insert, no method-specific rebalancing. Because the covering depends on the histogram, statistics matter more than for a GiST index – the area guard of Sect. 2.2 bounds the damage from a stale one, and ANALYZE on the 10-million-row corpora here takes 0.5–1.4 s.

3 The region index

Regions as stored objects. A footprint is covered by a small number of cells written in the NUNIQ encoding of the MOC standard and stored one per row in an ordinary B-tree. The footprints containing a given position are those whose cell is one of that position’s ancestors, a short list of equality lookups; refinement stops when cells become negligible next to the region, keeping the ancestor list short.

`skyregion` is one type internally tagged by kind rather than a family of per-shape types, so a single footprint column holds circles and polygons together – an ObsCore `s_region` column, say, mixes single-pointing circles with non-trivial polygons in the same rows, covered by one function regardless of kind. pgSphere has no equivalent: `scircle` and `spoly` are separate types, so a mixed-shape column needs two columns in its place.

3.1 A GiST opclass family for stored regions

A `skyregion` column can carry its own GiST index instead of the B-tree rewrite above, serving `&&` and both containment operators directly, keyed either by a small number of spherical caps (exact for a circle at any radius, but larger) or a plain axis-aligned 3D box. Shrinking the box key from `float8` (48 bytes) to outward-rounded `float4` (24 bytes, pgSphere’s own GiST representation) is sound because recheck is already `true` regardless of key precision: a looser box costs false positives the exact test removes, never rows. It is now the default region opclass; the multi-cap key remains better for a column mixing very different region scales, where box’s looseness around a large circle costs more than its smaller key saves. Measured against pgSphere’s native GiST on isolated single-scale bands, a mixed circle/polygon corpus, and the real `fpr` catalogue across all four region strategies: Sect. 6.6 (Table 10).

The same shrink does *not* transfer to a GiST key over `points` (`skypos_cap4_gist_ops`, shrinking only the internal-node cap, since a point’s leaf stores its position exactly and leaf exactness is load-bearing there): leaf entries dominate a GiST tree’s page count, and the leaf layer here has nothing to shrink, so no win results – the structural reason the region shrink is safe (recheck already `true`) does not hold for points. Numbers in Sect. 6.6.

4 What is proven, and what is tested

Proposition. Let R be a cone or convex polygon and let $\mathcal{C}$ be the covering computed for it. If (i) every point of a HEALPix cell at order k lies within $\text{maxpixrad}(k)$ of that cell’s centre, and (ii) every angle computed by the classification carries an absolute error below ϵ , then every order-29 cell that intersects R lies in one of the ranges of $\mathcal{C}$.

Proof sketch. The descent starts from cells whose union is the whole sphere and, at each step, keeps a cell, drops it, or replaces it by its four children (whose union is the parent), so the kept union shrinks only where a cell is dropped, which happens only when classified outside. With $R_{\text{cell}} = \text{maxpixrad}(k)$, (i) bounds every point of cell (k, p) within R_{cell} of its centre, so (ii) and the triangle inequality on the sphere make $\angle(\mathbf{q}, \mathbf{c}_{k,p}) - R_{\text{cell}} > r + \epsilon$ sufficient for the whole cell to miss the cone; the polygon case applies the same argument per bounding half-space. Truncation (range cap, area guard, step budget) only stops refinement, keeping a cell whole, so it cannot remove one either – a cell containing a point of R intersects R and is never dropped. $\square$

The proposition is conditional: (i) is a property of HEALPix, not of this work (the reference maxpixrad is analytic, verified over 1.6×10^6 sampled boundary points, largest observed ratio 0.9999; `skycell.exact_cells = off` rests on (i) alone, at the cost of Sect. 2.2); (ii) is an assumption about floating-point arithmetic, with ϵ chosen five orders of magnitude above the plausible accumulated error and five below the leaf cell.

Because a covering that misses a cell silently loses rows, the implementation is also checked by exhaustion: the HEALPix layer against known centres, round trips at all 30 orders, the nesting property, equal area ($\chi^2/\nu = 1.14$ over 768 cells), and the bounds of Sect. 2.2, over 1.6×10^6 sampled points including poles and zone boundaries; the covering against a 2-million-point clustered catalogue for cones to 120° and convex polygons, 30 000 adversarial cones (poles, zone boundary, face edges, radii $0.02 \text{ mas} - 10^\circ$) with 200 points each, and 7266 adversarial polygons with 866 537 points – all asserting zero false negatives, the test that caught the error of Sect. 2.2. Two SQL-level regression suites require every indexed answer to equal a sequential scan with the exact predicate (240 cones, 60 polygons, both join forms, MOC lookup; and the ADQL surface, index-plan presence, epoch propagation and frame conversions).

5 The ADQL surface

The extension exposes ADQL’s own vocabulary: types `skypos` and `skyregion` and functions `point`, `circle`, `box`, `polygon`, `contains`, `intersects`, `distance`, `area`, `coord1`, `coord2` and `centroid`, so a translator emits the standard’s spelling rather than a dialect, where pgSphere needs `spoint/scircle/spoly` and Q3C a separate fast path per shape with no region type at all; the operator form `pos <@ circle(...)` is the one a planner can answer from an index. Table 1 sets `skycell` against the pgSphere-plus-Q3C pair for the constructs where they differ most.

`skyregion`’s text representation is IVOA STC-S (Rots 2007) (`CIRCLE ICRS  $\alpha$   $\delta$   $r$  / POLYGON ICRS ...`), the same syntax an ObsCore `s.region` column already carries, so such a column casts directly from, and prints back as, the archive’s own text with no translation step. For hand-written SQL migrating from Q3C, `skycell_radial_query` and `skycell_join` take Q3C’s own argument lists as a prefix replacement, both reducing to the canonical `skycell_cone`.

`skycell` also implements the IVOA UDF registry’s epoch-propagation and frame functions (`ivo_epoch_prop`, `ivo_apply_pm`, `ivo_healpix_*`) with the rigorous propagation of the Hipparcos catalogue (ESA 1997), so a proper-motion-aware cone

Table 1: What a translator emits, for the ADQL constructs where the pgSphere-plus-Q3C pair and `skycell` differ most (the remaining surface – `BOX`, `DISTANCE`, `AREA/CENTROID`, epoch propagation – translates the same way and is omitted for space). `skycell` uses ADQL’s own spellings throughout; pgSphere needs region types and Q3C a separate fast path per shape.

ADQL	pgSphere + Q3C		skycell
<code>POINT/ CIRCLE/ POLYGON</code>	<code>spoint/scircle/spoly</code>		<code>point/circle/ polygon</code>
<code>CONTAINS(POINT, CIRCLE)</code>	<code>q3c_radial_query pos <@ scircle</code>	<i>or</i>	<code>point <@ circle</code>
<code>INTERSECTS(r₁, r₂)</code>	<code>r₁ && r₂</code> (no Q3C region type)		<code>r₁ && r₂</code>
<code>cross-match</code>	<code>q3c_join;</code> 2.0–3.8× (Sect. 6.5)	pgSphere slower	<code>skycell_cone</code> <i>or</i> <code>skycell_join</code> in the join

search is an indexable cone widened by the largest motion of interest, intersected with an exact test on the propagated position – exercised by round-trip tests (Barnard’s star to sub-microarcsecond over 16 years) rather than benchmarked.

6 Tests

6.1 Three corpora

Three corpora, each answering an objection the previous one invites. The *designed* corpus is 10 million synthetic sources with a density contrast resembling a real optical survey (35% uniform, 45% Galactic disk, 12% bulge, 8% in 200 clusters) – chosen by the method’s own authors. The *Gaia* corpus answers that without real positions: source counts from the ESA Gaia archive for every order-9 HEALPix cell (3 145 728 cells of 0.11° , summing to the published DR3 total) give 10 million positions placed proportionally to each cell’s count, the real sky’s crowding down to that map scale, Poisson below it by construction. The *real* corpus removes the remaining objection: ten million real Gaia DR3 positions, plus 1.6 million sources forming eight *complete* fields at full density (ω Cen, 47 Tuc, M4, M13, Baade’s Window, the Galactic centre, the LMC, and an off-cluster control), since thinning alone dilutes crowding to 0.55% of its true surface density – used for the density-estimator and ablation measurements (Sect. 6.4), where sub-cell structure is the whole question.

6.2 Protocol

Q3C 2.0.5, pgSphere 1.5.2 and `skycell` each hold the same logical rows in their own natural layout, all three physically ordered along a space-filling curve so heap locality is held equal (Table 2); row counts

are compared on every query. The measurement is organised as *trials*: one query centre and radius, all three methods answering it back to back in an order drawn at random (and recorded, so an order effect can be measured rather than assumed away), repeated five times with the query set traversed in random order each time. The analysis is paired – median over repetitions per method, ratio between methods for that query, so its own difficulty cancels – with a percentile bootstrap (4000 resamples) interval; a ratio is called a difference only when its interval excludes 1. Everything is measured *warm* (an untimed pass precedes the measurement) and *cold* (server restarted, caches empty, first repetition only), with a separate randomized EXPLAIN (ANALYZE, BUFFERS) pass recording planning time, execution time, buffer pages and row estimate apart from each other. Cone radii span 1'' to 3° with 16–400 queries each, half from catalogue positions and half uniform on the sky; cross-matches join a 200 000-row probe table to the catalogue, half displaced by $\sim 0.3''$; footprint queries ask which of 20 000 stored circles contain each probe.

6.3 Reproducibility

Table 2 records the two configurations in enough detail to repeat or contest the measurements.

What this protocol still does not control.

Two machines, each with one storage device, sharing one configuration – absolute times are properties of a machine, only the comparisons travel. The custom scan was measured only on the second, so comparing it with the range rewrite across tables also compares hosts and CPU architectures; where the two ran in the same trials (the rewrite’s ratios in Sect. 6.4, its columns in Table 7) they do not. The three extensions are compared as configured here: a different Q3C formulation or pgSphere index configuration could move their numbers (Sect. 6.4 reports one such effect dominating Q3C’s small-cone results).

6.4 Source-like catalogues

Cone searches. Table 3 and Fig. 1a give the primary result: paired ratios with intervals, on both corpora, warm and cold.

These two columns are *skycell*’s custom scan, its default (Sect. 2.3). Warm, it is faster than pgSphere at every radius on both synthetic corpora, by 17–33% (ratios 0.67–0.83), every interval excluding 1. The range rewrite, run in the same trials on the same host, is 0.88–1.07 of pgSphere up to 30' and 0.69–0.91 at 1° and 3° – where the original measurement on the host of Table 2 put it – so the gain is the custom scan’s, not the host’s: 0.78–0.84 of the rewrite’s time up to 30', 0.87–0.97 above (*bench/results-pg18/rewrite-vs-custom.txt*).

Table 2: The measured configurations. The second host ran the custom-scan measurements, with the range rewrite alongside in the same trials; everything else is from the first. Rows spanning both columns are common to the two.

Item	First host	Second host
Machine	Apple M3 Pro laptop (aarch64), 11 cores; Docker Desktop Linux VM, 8 GB shared with other containers; NVMe SSD	cloud micro-VM (KVM, x86-64), 4 vCPU Intel Xeon 2.8 GHz, 15 GB; one virtual disk, its backing storage not visible
PostgreSQL	18.6, PGDG package	18.6, built from source (gcc 13.3, -O2, no LLVM)
Settings	shared_buffers 2 GB, effective_cache_size 5 GB, random_page_cost 1.1, work_mem 64 MB, JIT off, parallelism off	
Extensions	Q3C 2.0.5, pgSphere 1.5.2, skycell 0.6 (range rewrite)	the same Q3C and pgSphere built from source; skycell 0.26 (custom scan and range rewrite)
Range price	Eq. (7), derived per relation, 116 here	Eq. (8) for the custom scan, 34 on the Gaia catalogue; Eq. (7) for the rewrite, 110
Used for	every table and figure not listed opposite	synthetic columns of Table 3, <i>custom</i> columns of Table 4, Tables 7 and 8, Fig. 1a,b
Rows	10 ⁷ per corpus (5 × 10 ⁷ for the Gaia corpus’s scale test, first host only)	
Schema	each extension’s own B-tree/GiST on its own key (q3c_ang2ipix , spoint , cell_int8), heap ordered by its own key; VACUUM (FREEZE) and ANALYZE (stats target 1000) before measuring	
Repetitions	5 (warm), 1 (cold); 1456 queries per repetition	
Uncertainty	percentile bootstrap, 4000 resamples, over queries	

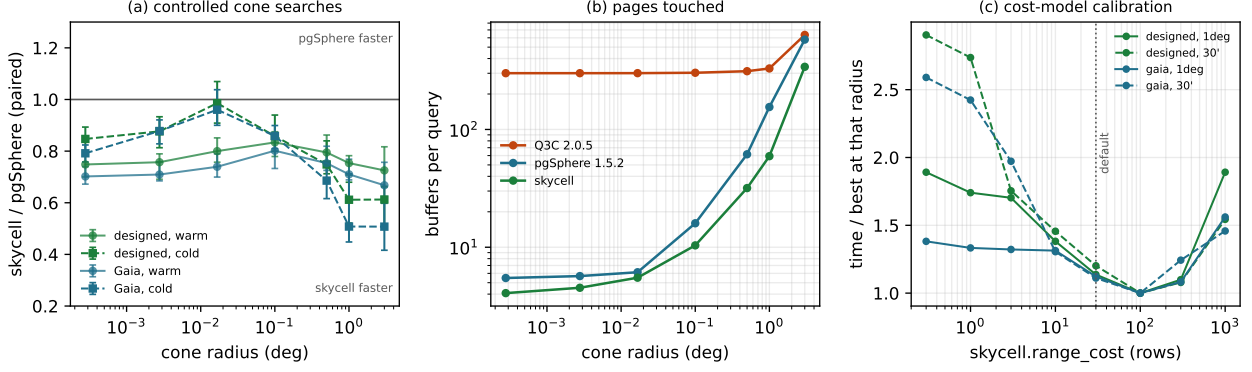


Figure 1: (a) Paired ratio of **skyCell** to **pgSphere** against cone radius, 95% bootstrap intervals, both corpora and cache regimes; below the line **skyCell** is faster. (b) Buffer pages touched per query (mean, designed corpus): at 1° **skyCell** reads a third of **pgSphere**’s pages. (a) and (b): **skyCell**’s custom scan, the trials of Table 3’s synthetic columns; (c): the range rewrite on the host of Table 2. (c) The cost model’s one free parameter, swept over four decades and normalised to the best setting per radius; flat-bottomed over a decade either side of its minimum, why deriving it (Sect. 6.4) costs nothing in speed.

At 1'' it plans in 0.036 ms against the rewrite’s 0.055, still above **pgSphere**’s 0.029, and wins in execution, 0.022 ms against 0.040, reading 4.1 pages on average to **pgSphere**’s 5.5. This matters more than it sounds: for the range rewrite on the host of Table 2, measuring one method’s whole grid then the next instead gives ratios of 0.69–0.87 below 30' where the paired protocol finds 0.89–1.03, since the phase-ordered arrangement flatters whichever method runs later – here, **skyCell**. The paired number is the one we stand behind.

Cold, the custom scan is faster at every radius on both synthetic corpora except 1', where the two are level (0.96–0.99): by 12–21% at 1'', 10'' and 6', 25–31% at 30' and a factor 1.6–2 at 1° and 3°, because it touches a third of **pgSphere**’s buffer pages at 1° (on average 59 against 155 on the designed corpus, 85 against 282 on the Gaia one) – a few microseconds of memory traffic when those pages are warm, the whole cost of the query when they are not. Cold ratios depend on the storage under the page cache, which differs between the two hosts; the original cold measurement of the rewrite (the 10⁷ rewrite columns of Table 4) found larger margins at 30' and 3° (0.57 and 0.40) and about the same at 1°.

On real Gaia DR3 positions, warm, the small-cone advantage reverses. The resampled corpus places positions uniformly within each 0.11° map cell; the real one keeps every scale of clustering the map cannot see. On 10 million real DR3 positions (Table 3’s rightmost column), **skyCell** is 12–45% *slower* than **pgSphere** from 1'' to 30', ties at 1°, and is 27% faster at 3° – the one radius where the resampled corpus’s number still holds. Buffers rule out the data itself as the cause: **skyCell** touches *fewer* pages than **pgSphere** at every radius (0.74 of

pgSphere’s at 1'', 0.27 at 1°, the same as the rewrite’s 0.27 on the resampled corpus), and execution time alone is faster or level everywhere (0.59–0.99). The whole deficit is **skyCell**’s own planning – the covering computation and rewrite, 0.017 ms per query at 1'' rising to 0.25 ms at 3° – measured against **pgSphere** answering a small cone in about 0.05 ms total, so a few hundredths of a millisecond of planning is not lost in the noise the way it is at larger radii. This is a genuine limitation, not an artefact of the resampled corpus’s own structure: real sub-cell clustering does not cost **skyCell** extra rows fetched, it costs the time spent deciding how to cut the cone before any row is touched, and that cost does not scale down with the query. Cold, the picture divides by radius (Table 3, last column). Here the two methods read separate copies of the same rows, so neither warms the other’s pages; every centre is new and kept apart from every other; and the server is restarted before each method’s pass at each radius. From 30' up **skyCell** is faster, 0.89 at 30' and 0.62–0.65 at 1° and 3°, because it reads 0.5–0.7 of **pgSphere**’s pages and those reads now dominate. Below that the verdict is mixed (1.06 at 1'', 0.91 at 10'', 1.21 at 1', level at 6'): a first-touch query there spends 0.2–0.4 ms reading 2–6 pages, which swamps the 0.03 ms planning difference that decides the warm result, and **skyCell**’s one fewer page read is not reliably worth more than that difference. These reads cost about 0.08 ms each: the container’s page cache was dropped, but a cache on the host beneath it could not be, so this is cold for PostgreSQL and the virtual machine, not for the disk. With true disk latency each page saved would count for more, and the small-radius ratios should move towards **skyCell**’s page ratio (0.7–0.9 there) – expected, not measured here. These real-position columns are the range rewrite, measured before the

custom scan existed, and the warm deficit they show is planning; on the synthetic corpora the custom scan removes about a third of `skycell`'s planning at the smallest radii (0.055 to 0.036 ms at 1''), more than that deficit. Whether it turns the real-position result is not measured: the real corpus could not be rebuilt where the other two columns were.

Against Q3C, `skycell` is faster by a factor 1.7–50, a comparison of *implementations*: `q3c_radial_query` 2.0.5 expands every call into 100 key ranges regardless of radius, about 300 buffer pages before any row is read. Against `q3c_join`, intended for cross-matching and carrying no such fixed expansion, the rewrite's join is level with it and the custom scan's 25–33% faster (Sect. 6.5) – informative about the two indexing *approaches*; about the cube quad-tree as a method we make no claim, since a covering-aware radial query in Q3C would plausibly close most of the gap. Neither method was given prepared statements or plan reuse, which would help `skycell` most. The order effect the randomization controls is small but real, inflating variance rather than biasing the ratios.

Does the advantage grow with the catalogue?

Ten million rows is 0.55% of Gaia DR3, so we repeated the protocol for the range rewrite, on the host of Table 2, on a 50-million-row Gaia-density corpus, where pgSphere's 3442 MB GiST index exceeds the 2 GB `shared_buffers` while `skycell`'s 1072 MB B-tree still fits. The answer (Table 4) divides by region size: for 30' and larger the advantage grows markedly (at 1° the warm ratio improves from 0.85 to 0.60, cold from 0.53 to 0.31, as the buffer gap widens from 55/154 to 152/514), while cones of an arcminute and below do not move, since both methods touch five to seven pages regardless of table size and only `skycell`'s planning grows with it. For the rewrite the rule is about pages rather than rows: an arcsecond cone on any catalogue is not the workload it improves; a degree-scale selection, a polygon or a cross-match on a large one is. The custom scan changes the small-cone half of that rule, not the large-cone half: at 10^7 rows (rightmost columns, a different host) it is 0.70–0.74 of pgSphere warm at an arcminute and below, where the rewrite stays at 0.89–1.02 at both sizes; at 30'–3° it is 0.67–0.75 warm against the rewrite's 0.69–0.92 at the same size, and mixed cold, across the change of host (0.51–0.69 against 0.40–0.57). Whether the custom scan keeps the rewrite's gain with size is not measured: the 50-million-row corpus did not fit the second host's disk.

Index size, build time, and what they cost at Gaia's scale. Index size and build time are operational numbers worth carrying to the scale an archive runs at (Table 5). The cost per row is flat between 10^7 and 5×10^7 rows, so extrapolating to Gaia DR3's 1.81×10^9 sources compares *deployments*: an archive

wanting both cone searches and cross-matching carries pgSphere plus `q3c_join` (2.4 times faster than pgSphere at the latter, Sect. 6.5) today; `skycell` answers both from one, smaller index.

The memory column is the operational point: at DR3 scale a 64 GiB server holds `skycell`'s 38 GiB index with room to spare while the pgSphere-plus-Q3C pair needed for the same coverage wants 256 GiB. That factor-four gap is a procurement decision at these sizes, and it lands where our cold-cache measurements put `skycell` furthest ahead: warm `skycell` against cold pgSphere on the 5×10^7 -row corpus – the comparison a 64 GiB archive actually faces, since only the former's index is resident there – is one to two orders of magnitude apart at 30' and beyond.

This is an upper bound: the extrapolation is linear and validated only over a factor five in row count, covers the positional index alone (dwarfed in a real archive by the table and its other indexes), and a partitioned deployment needs only its active partitions resident. Since the two B-trees are the same size, the real saving is not needing the geometry index at all, which rests on `skycell` answering the region queries pgSphere is kept for (Sects. 6.4, 6.5).

The cost model: its parameter, its input, and what it is worth. Equation (2)'s one free parameter, c_{range} , need not be a parameter at all: it is a ratio of two costs PostgreSQL already models, one more index range (a B-tree descent) against one false-positive row (its index entry, heap tuple, exact predicate, and heap page amortised over the rows sharing it):

$$c_{\text{range}} = \frac{(\lceil \log_2 N \rceil + H + 1) 50 c_{\text{op}}}{c_{\text{tuple}} + c_{\text{itup}} + k c_{\text{op}} + c_{\text{page}}/n_{\text{pp}}}, \quad (7)$$

with $n_{\text{pp}} = \text{reltuples}/\text{relpages}$. Every input is already at hand, so `skycell.range_cost` defaults to evaluating it per relation (116 on the catalogues; 93 on a wider-row ObsCore relation). Placed in the same trials as the fixed constant it replaces, the two are indistinguishable, since the cost curve is flat (sweeping four decades moves the median time under 25% between 3 and 300): the derivation buys not speed but that the parameter stops being a number someone has to choose, and the order it picks is likewise close to optimal (about 3% against one order finer at 6') for the same reason.

Equation (7) is the rewrite's price, and knowingly high: it charges the descent's comparisons at 50 times what `btcostestimate` does and every page as a random read, which stands in for the per-range planning the rewrite incurs and Eq. (2) does not model. The custom scan plans no range, so it prices one as it pays for it: the descent as `btcostestimate` charges it, plus two pages – the leaf the descent lands on and the heap page where

Table 3: Cone searches: paired ratio of **skycell** to pgSphere, 95% percentile-bootstrap interval over queries. Below 1 means **skycell** is faster. **Bold/italic**: interval excludes 1, faster/slower; plain: not distinguishable. n : distinct queries at that radius, each answered five times per method (warm) or once (cold), all methods randomized per trial (the skycell/Q3C ratio, 0.02–0.58 depending on radius and cache regime, is given in the text, Sect. 6.4). *Designed* and *Gaia*: **skycell**’s custom scan, its default (Sect. 6.4), PostgreSQL 18.6 in a 4-core cloud container, **bench/results-pg18/**; cold there is the server stopped and the container’s page cache dropped before a single pass. *Real*: the range rewrite on the host of Table 2, measured before the custom scan existed; 10 million real Gaia DR3 positions (**random_index** < 10⁷), its own centre draw at the same radii and counts, total time. Warm: one shared heap, per-method warm-up. Cold: one table per method, fresh centres, server restarted before each method’s pass at each radius, first touch only; cold for PostgreSQL and the virtual machine but not for the host’s own cache. Sect. 6.4 explains why these columns move against the other two, and why they are not yet a test of the custom scan.

radius	n	custom scan, PostgreSQL 18.6, cloud container				range rewrite, Table 2			
		designed corpus		Gaia corpus (resampled)		real corpus		warm	cold
		warm	cold	warm	cold	warm	cold		
1''	400	0.75 [0.70, 0.79]	0.85 [0.80, 0.89]	0.70 [0.67, 0.74]	0.79 [0.76, 0.83]	<i>1.12</i> [1.10, 1.16]	<i>1.06</i> [1.01, 1.11]		
10''	400	0.76 [0.69, 0.79]	0.88 [0.81, 0.94]	0.71 [0.68, 0.76]	0.88 [0.83, 0.92]	<i>1.16</i> [1.14, 1.19]	0.91 [0.87, 0.95]		
1'	300	0.80 [0.76, 0.85]	0.99 [0.91, 1.07]	0.74 [0.70, 0.79]	0.96 [0.90, 1.04]	<i>1.23</i> [1.19, 1.26]	<i>1.21</i> [1.11, 1.27]		
6'	200	0.83 [0.78, 0.88]	0.86 [0.80, 0.94]	0.80 [0.74, 0.86]	0.86 [0.81, 0.90]	<i>1.45</i> [1.38, 1.49]	0.95 [0.91, 1.04]		
30'	100	0.79 [0.75, 0.86]	0.75 [0.71, 0.84]	0.75 [0.73, 0.82]	0.69 [0.62, 0.76]	<i>1.29</i> [1.12, 1.47]	0.89 [0.81, 0.98]		
1°	40	0.75 [0.71, 0.78]	0.61 [0.52, 0.68]	0.71 [0.69, 0.77]	0.51 [0.45, 0.72]	1.00 [0.89, 1.26]	0.65 [0.58, 0.70]		
3°	16	0.73 [0.66, 0.82]	0.61 [0.50, 0.66]	0.67 [0.63, 0.76]	0.51 [0.41, 0.60]	0.73 [0.62, 0.85]	0.62 [0.49, 0.82]		

Table 4: The same protocol at two catalogue sizes on the Gaia density field: paired ratio of **skycell** to pgSphere, 95% bootstrap intervals. *Rewrite*: the range rewrite on the host of Table 2, at both sizes; at 5×10^7 rows pgSphere’s index (3442 MB) exceeds **shared_buffers** while **skycell**’s (1072 MB) does not. *Custom*: **skycell**’s custom scan at 10⁷, PostgreSQL 18.6 in a 4-core cloud container (the Gaia columns of Table 3); the 5×10^7 corpus did not fit that container’s disk. **Bold**: **skycell** faster; *italic*: slower; both excluding 1.

radius	warm		cold		10 ⁷	5 × 10 ⁷
	rewrite	custom	rewrite	custom		
	10 ⁷	5 × 10 ⁷	10 ⁷	5 × 10 ⁷		
1''	0.89	0.89	0.70	0.78	0.71	0.79
10''	0.90	0.94	0.71	0.80	0.88	0.88
1'	0.93	1.02	0.74	0.95	0.95	0.96
6'	<i>1.03</i>	1.00	0.80	0.90	0.89	0.86
30'	0.92	0.72	0.75	0.57	0.49	0.69
1°	0.85	0.60	0.71	0.53	0.31	0.51
3°	0.69	0.58	0.67	0.40	0.23	0.51

Table 5: Index cost at 10 million rows and extrapolated to Gaia DR3 scale (1.81×10^9 sources, linear from the 5×10^7 measurement; per-row cost is flat to 0.5% between the two measured scales). Last column: smallest conventional server memory holding the index resident with room for the heap.

index	10 ⁷ rows		at Gaia DR3 scale		
	MB	build (s)	GiB	build	mem GiB
Q3C	214	2.2	38	13 min	64
pgSphere	685	42.3	122	2.7 h	192
skycell	214	1.4	38	14 min	64
pgSphere + Q3C	—	—	160	2.9 h	256

the scan stops running sequentially – each priced at $p = f 50 c_{\text{op}} + (1 - f) c_{\text{page}}$ by where it is expected to come from,

$$c_{\text{range}}^{\text{cs}} = \frac{\lceil \log_2 N \rceil c_{\text{op}} + 50(H + 1) c_{\text{op}} + 2p}{c_{\text{tuple}} + c_{\text{itup}} + k c_{\text{op}} + p/n_{\text{pp}}}, \quad (8)$$

with $f = \min(1, \text{shared_buffers}/\text{relpages})$ the fraction of the relation the buffer pool can hold: a page in the pool costs what touching an index page does, one outside it a random read. For the 10-million-row Gaia catalogue of the custom-scan runs (**random_page_cost** = 1.1, 120 rows per page) this is 34 rows with the relation in the pool and 87 with none of it there, against 110 from Eq. (7) for the same relation, so the custom scan cuts its coverings finer.

The model’s input, $\hat{\rho}$ from Eq. (5), was called unbiased in an earlier version of this work – true of positions resampled from a density map with no structure below 0.11°, but not of real Gaia DR3 positions. Table 6 compares $\hat{\rho}$ with ρ counted from the same relation: within 10% at Baade’s Window and a factor 1.6 at the Galactic pole (where the sample holds a few dozen rows), *low by a factor 10–100* in a globular cluster core, *high by a factor 3.7* at the Galactic centre, as the histogram resolves a cluster while a query sits inside it and averages it away once the query is larger (the failure Sect. 2.2 predicts, now measured on real positions). The estimate also moves between **ANALYZE** runs, which sample the relation afresh: at 47 Tuc the median ranges over 0.01–0.08 across ten. The consequence is smaller than the error: since $s^* \propto \sqrt{\rho}$, an order of magnitude in ρ is a factor three in cell size, and the cost curve is flat across much of that – the covering chosen in a cluster core is within 0.96–1.55 of the best fixed or-

Table 6: The density estimate against the truth, real Gaia DR3 positions: $\hat{\rho}$ is what the cost model reads, ρ counted from the same relation, both in rows sr^{-1} , over four decades of surface density, radii 0.01° – 1° . $\hat{\rho}/\rho$: median over radii, then median over ten ANALYZE samples; *samples*: spread of that median across them; *range*: over all radii and samples. `bench/29_estimator.sql`, PostgreSQL 16.

field	$\hat{\rho}/\rho$	samples	range
ω Cen	0.07	0.06–0.08	0.05–0.56
47 Tuc	0.03	0.01–0.08	0.00–0.94
LMC	0.69	0.42–0.85	0.22–1.16
Baade’s Window	0.96	0.92–0.99	0.47–1.14
Galactic pole	1.31	1.18–1.58	0.49–1.80
Galactic centre	3.69	3.48–3.83	1.99–5.47

der (1.76 in a sample that put 47 Tuc a hundredfold low), against 0.96–1.25 where the estimate is accurate, and against 243–830 for a fixed order chosen badly. The error costs something in cluster cores at intermediate radii; it does not cost a wrong covering. The estimator should be judged by the decision it produces, not the number it reports.

The comparison with pgSphere confounds three things, separated on the real Gaia corpus: *heap clustering* is a large absolute effect but almost no differential one (ratio 0.64 clustered against 0.60 unclustered at 1°); *adaptive resolution* is worth a great deal (against every fixed order 4–13 the penalty for choosing badly is 243–830 $\times$, while the cost model lands within 1.02–1.58 of the best without knowing the answer); and *the index representation* is isolated by comparing against Q3C instead, both B-trees over an `int8` key differing only in covering strategy (Sect. 6.4, with the caveat of Sect. 6.4).

The service-level A/B, and where the crossover is. A benchmark of index operations is not a benchmark of an archive. We ran the same build inside *egernia* (Delli Veneri & Salgado 2026), a TAP 1.1 server, against its published 500 096-row ObsCore corpus: one database holding both indexes, two otherwise identical service processes differing only in how they translate CONTAINS, interleaved over six query classes at 1, 8 and 32 concurrent clients. Both targets pass the conformance and cross-server agreement gates on all eleven portable classes, identical rows through both, and the performance result is a tie (throughput ratios 0.96–1.06) although at the database level *skycell* is 1.2–1.45 times *slower* per query: a cone over half a million rows costs a fraction of a millisecond either way against roughly 10 ms of CPU in the service’s own Python, so the database cannot move throughput. That explanation predicts a crossover at larger relations, checked by building the same ObsCore-shaped relation (observations in groups of 128 sharing a field centre, clustered by collection and field) at three sizes and

asking each the same 30 queries (Table 7). Field and query centres are a stored set drawn once from the DR3 density map (`bench/data/`), and the relation at a given size is the same on every run. Earlier revisions of this table took the field centres from the first rows of whichever catalogue corpus was loaded; the resampled-Gaia corpus is generated in HEALPix order, so those rows were one patch of sky and every observation was packed into it (a 2° cone returned 1.2×10^5 rows at 10^7 , against 8×10^3 now), and the result moved with the corpus – the irreproducibility those revisions reported, not a property of either index. Like the synthetic columns of Table 3, it was measured with *skycell*’s custom scan (Sect. 6.4), and the range rewrite alongside, on PostgreSQL 18.6 in a 4-core, 15 GB cloud container, not the host of Table 2. The predicted crossover is there. The small cone (Q05) is 1.14 times slower than pgSphere at 5×10^5 rows and at parity by 10^7 ; the cone with catalogue cuts (Q07) is at parity up to 2×10^6 and 20% faster at 10^7 ; the degree-scale cone (Q06) is never slower and 25% faster at 10^7 , where pgSphere reads 939 buffer pages to *skycell*’s 591; the empty cone (Q12), a few hundredths of a millisecond either way, is faster at every size. Run with the range rewrite instead, the same queries reach nearly the same ratio on Q06 at 10^7 (0.78), but the small cone stays 2.1–3.5 times slower and Q07 does not cross: for degree-scale cones the margin is the index, for the small ones it is the custom scan, which at 10^7 plans Q05 in 0.024 ms against the rewrite’s 0.086 and executes it in 0.091 ms against 0.168: the rewrite’s plan is a `BitmapOr` with an index descent and a bitmap per range, where the custom scan walks all its ranges into one bitmap. The three translations return identical rows for every query, size and run.

When the index no longer fits in memory. Tested directly in a 3 GiB container (768 MB shared buffers) holding a 22 GB, 20.5-million-row ObsCore-shaped relation (GiST index $1.51 \times$ the buffer pool, *skycell* $0.57 \times$): *skycell* answers 4.2–4.8 times faster over 3500 uniform 2° cones, serving 53% of its index accesses from the pool against pgSphere’s 9%. The 22 GB heap takes most of the pool regardless, so neither index is fully resident – this measures a smaller index winning a contested pool – and interleaving both translations in one pass instead of separate phases (each then leaving the heap warm for the other) puts the gap at $1.6 \times$ rather than 4.2.

6.5 Crossmatches

Convex polygons of 0.05° – 2° take a median 0.281 ms on the Gaia corpus against pgSphere’s 0.767 ms and Q3C’s 1.864 ms, similarly on the designed corpus: a factor 2.7–4.4 either way, with pgSphere’s mean and 95th percentile far above its median where a polygon’s bounding volume is a poor fit.

Table 7: Crossover on an ObsCore-shaped relation, randomized paired trials (30 stored centres, the same at every size; each method warmed on its own query first): total query time as the ratio to pgSphere, pooled over two complete runs of 3 repetitions that differ only in trial order, for `skycell`’s custom scan (its default) and its range rewrite (pgS.: pgSphere); median buffer pages at 10^7 rows. Q05: 0.15° cone (median 126 rows at 10^7); Q06: 2° (8350); Q07: 0.5° with calibration/time cuts (502); Q12: an empty $0.7''$ cone. Between the two runs the custom-scan ratio differs by at most 0.19 (Q07, 5×10^5), and by 0.02 or less at 10^7 except Q12 (0.12). PostgreSQL 18.6, 4-core cloud container (text). `bench/10.crossover.sql`, `bench/results-pg18/crossover-stored/`.

rows (10^6)	custom scan / pgS.		rewrite / pgS.		buffers, 10^7		
	0.5	2	10	0.5	10	pgS.	skycell
Q05	1.14	1.07	0.97	3.52	2.14	32	32
Q06	0.92	0.93	0.75	1.30	0.78	939	591
Q07	0.94	0.99	0.80	1.89	1.05	113	89
Q12	0.71	0.68	0.47	1.01	0.79	7	3

Cross-matching a user’s uploaded target list is the operation the Gaia archive lists among its server-side capabilities (Salgado et al. 2017) and Q3C’s own speciality (`q3c_join`), so it deserves the cone searches’ protocol, with each method warmed on its own query immediately before timing to avoid a spurious I/O-ordering effect at these sizes (14 GB against 2 GB of `shared_buffers`). In `skycell` it is the same join written either way, `JOIN cat c ON skycell_cone(c.cell, c.ra, c.dec, p.ra, p.dec, r)`, answered by the range rewrite’s fixed slots or by the custom scan’s covering per probe (Sect. 2.3). On the first host, at 5×10^7 rows and with the rewrite, `q3c_join` is the fastest established method, 2.0–2.6 times faster than pgSphere, and `skycell`’s join is not distinguishable from it at $1''$ or $10''$ (1.04 and 1.07, two runs pooled), while 2.0–2.5 times faster than pgSphere. On the second host, at 10^7 rows (Table 8), the custom scan makes the same join 25–29% faster than `q3c_join` and 3.2–3.7 times faster than pgSphere, returning the same rows as `q3c_join` in every trial; run through the rewrite in the same trials it is again not distinguishable from `q3c_join` (0.99–1.17). Written as `skycell_join`, the spelling a `q3c_join` user would write, over an expression index on `skycell_ang2cell(ra, dec)`, it is the same node at the same speed (0.96–0.97 of the `skycell_cone` form, not distinguishable), 27–33% faster than `q3c_join`. The difference is per-probe work, not the sky scanned: the rewrite’s plan is fixed at four range slots per probe, a `BitmapOr` whose every arm descends the index whether the probe’s covering filled it or not (12.6 buffer pages per probe in one block at $1''$), where the custom scan walks the

Table 8: Cross-matching 200 000 probes against the 10^7 -row resampled-Gaia catalogue on the second host (Table 2): paired ratio over 16 trials (8 blocks of 25 000 probes, twice), 95% bootstrap interval. *Cone*: the join written with `skycell_cone` on the stored cell column; *join*: written as `skycell_join`, Q3C’s spelling, on an expression index; *custom*: answered by the custom scan; *slots*: through the range rewrite. **Bold**: interval excludes 1. `bench/11.xmatch.ab.sql`, `bench/results-pg18/xmatch/`.

	$1''$		$10''$	
against <code>q3c_join</code>				
cone, custom	0.71	[0.64, 0.79]	0.75	[0.69, 0.86]
join, custom	0.67	[0.60, 0.77]	0.73	[0.63, 0.84]
cone, slots	0.99	[0.85, 1.15]	1.17	[0.94, 1.40]
against pgSphere				
cone, custom	0.27	[0.23, 0.30]	0.31	[0.24, 0.35]
join, custom	0.26	[0.23, 0.30]	0.31	[0.26, 0.33]

probe’s own covering, 1.2 ranges on average, in one node (4.3 pages). The covering itself is no tighter – the sky it scans, as a multiple of the sky asked for, still peaks around 105 near $0.25\text{--}1''$, since the cost model won’t cut finer where extra ranges cost more than the area saved, and falls to 1.07 above 30° .

We repeated the rewrite’s measurement on the real corpus, on the first host, at the radii cross-matching actually uses – optical $1\text{--}1.5''$, radio up to $5''$ (Table 9; an earlier $30''$ point was an association radius, not a cross-match one, dropped). Over that range the join through the rewrite is 1.32 of `q3c_join` and 2.3 times faster than pgSphere – the fixed slots’ per-probe cost again, which on the synthetic corpus is what the custom scan removes. The custom scan has not been measured on real positions (the real corpus could not be rebuilt on the second host), so the claim is bounded by that: on the operation Q3C is built for, `skycell`’s join is Q3C’s better on the resampled corpus and behind it on real positions through the rewrite, and several times faster than pgSphere throughout, with the same index answering the region queries of Sect. 6.4. Stored footprints follow the same pattern: the MOC-in-a-B-tree arrangement answers 200 474 point-in-region pairs in 0.76 s against pgSphere’s 0.97 s, with a larger, slower-to-build index since a region becomes several rows rather than one.

Estimates, plans and joins. A spatial predicate is rarely alone: three shapes (a cone with a catalogue cut, joined to an observation table, and a three-way join) through each method show `skycell`’s estimate advantage carrying over (median error 1.15–1.88 against 1.63–2.33 for Q3C and 2.50–3.00 for pgSphere), but the plans are the same – all three chose the same strategy in 134 of 135 queries. Better estimates are demonstrated, better plans are not: an accurate estimate is insurance against a query we

Table 9: Cross-matching at the radii the operation is used at, on **real Gaia DR3 positions**, first host, **skycell**’s join through the range rewrite: geometric mean of the paired per-cell ratio over three upload sizes and two target distributions. Below 1: **skycell** faster.

radius	vs q3c_join	vs pgSphere
0.5''	1.26	0.46
1''	1.12	0.45
1.5''	1.66	0.42
5''	1.29	0.42
all	1.32	0.44

Table 10: Region-opclass wall-clock, real 50 000-row **fpr** catalogue, 500 probes/strategy: ratio of the **float4** box (**skyregion.box4.gist_ops**, now default) to pgSphere’s native GiST. Below 1: **skycell** faster.

strategy	box4 / pgSphere
&& (overlap)	0.12–0.17
@> (region,point)	0.75–0.93
@> (region,region)	0.55–0.70
<@ (region,region)	0.26–0.27

did not write, and we have not shown that insurance paying off on one we did.

6.6 Regions

On isolated single-scale bands (0.05° – 1°), the **float4** box key (**skyregion.box4.gist_ops**, Sect. 3.1) beats pgSphere’s native GiST by 10–25% fewer buffers and 10–20% less time; on a mixed circle/polygon corpus the margin widens (pgSphere pays for two index descents per probe there, one per region kind), and on the real 50 000-row **fpr** catalogue, across all four region strategies (Table 10), the **float4** box wins every one – including point containment, the one strategy its own **float8** predecessor still loses to pgSphere on.

For points, the same shrink buys nothing: buffers are statistically indistinguishable from the unshrunk cap key at every radius from $1''$ to 3° , and pgSphere’s native point GiST still wins by a stable 1.4 – $2.7\times$ from $1''$ through $30'$, converging within noise by 1° – unchanged by the shrink, since the structural reason region leaves could shrink safely does not hold for points (Sect. 3.1).

7 Conclusions

The advantage is not universal, and most of what is universal about it is recent: warm, the range rewrite alone is within 12% of pgSphere below $30'$, and the 17–33% margin at every radius on the synthetic corpora is the custom scan’s, measured on a second host and not yet on real positions (Sect. 6.4); on an ObsCore-shaped relation of 500 000 rows it is slower on the small cone, and reaches pgSphere only by 10^7

rows, which it does only through its custom scan: the range rewrite stays 2.1–3.5 times slower on that cone at every size measured (Sect. 6.4). Better row estimates did not, on the join shapes we wrote, produce better plans. Every number comes from one machine and one storage device, except the custom scan’s, which come from a second (Tables 3, 4, 7 and 8), with a density field standing in for real positions except where Sects. 6.5, 6.4 and the real-position paragraphs of 6.4 say otherwise; on those real positions the range rewrite’s small-cone result is a clean loss warm and a mixed result cold, not the win the other two corpora show, and the custom scan is untested there.

What remains expensive is planning: a covering costs 0.015–0.12ms even cached against 0.006–0.008ms for a GiST predicate needing no such step, the whole of **skycell**’s disadvantage at $6'$. Three attempts to cut this were tried; two did not pay off. An SP-GiST class over the cell hierarchy, tried to remove it architecturally, costs correctness margin elsewhere (the duplicate-position tolerance shrinks by roughly an order of magnitude), so it stays exploratory. Letting the covering search try one or more orders finer than Eq. (3)’s closed form and keeping whichever scores lowest (**skycell.probe_orders**, off by default) does pick a measurably tighter covering – worth 30–48% in *execution* time at several radii. Measured in *total* (planning plus execution) time against query centres that do not repeat – the realistic case for a service answering different users’ cones, where a per-backend cache cannot amortise the extra enumeration – the gain reverses at exactly the radii it looked largest, since trying a finer order costs roughly four times the cell classifications already spent, paid at plan time on every query. We found and fixed a latent bug this exposed (the probe loop’s own cost accounting shared a field with an unrelated guard in the general descent, so tuning one silently detuned the other) but could not support promoting the tuning itself on the evidence in hand – a caution about measuring a cost-based index’s own overhead, not only what it saves.

The third, narrower, did pay off. The per-order chord bounds a covering’s classify step reads were computed for all thirty HEALPix orders on every call, regardless of how many a given covering actually visits – typically one or a handful. Filling them lazily, only for orders actually visited, changes no covering and no buffer count at any radius tested. Measured on a different host from the rest of this paper, via a same-session randomized A/B against the unmodified build sharing one warm cache and one exact-test function (so only the covering computation itself differs), it cuts planning time by about 1% pooled across **tab:cones**’ own radii (95% bootstrap CI excluding 1; no radius resolves a loss) – real, but an order of magnitude smaller than measuring the covering computation alone in isolation

suggested (7–22%), since planning time also spends cycles on statistics-cache and syscache lookups this change does not touch. An index descent avoiding planning cost altogether remains open.

The density synopsis is the other limit: an equi-depth histogram cannot see a cluster smaller than one of its buckets, and raising the statistics target does not help, since a denser region gets narrower buckets and the interpolation stays scale invariant. The area guard bounds the damage; a `typanalyze` storing a multi-order count map would estimate what the covering needs, implemented as an option with no measurement yet showing it helps.

`skycell` keeps Q3C’s compact integer key and B-tree while answering the region queries `pgSphere` was built for, treating a covering’s depth as a cost question from local source density and the price of an index range. Against `pgSphere` it is faster for cone searches at most radii warm and at every radius cold (up to $2.5\times$) on the designed and resampled-Gaia corpora; on real, unresampled Gaia DR3 positions this reverses below 1° warm and is unsettled below $30'$ cold, a genuine limit of the cost-based planning step rather than of the index (Sect. 6.4). It also answers convex polygons $2.7\text{--}4.4\times$ faster; at the cross-match Q3C is built for, its custom scan is 25–33% faster than `q3c_join` on the resampled corpus (on real positions only the slower range rewrite has been measured) and $3.2\text{--}3.8\times$ faster than `pgSphere`, with an index a third the size (38 against 160 GiB at Gaia DR3 scale for the `pgSphere-plus-Q3C` pair), built thirty times faster, from a cost model whose one parameter is derived rather than configured. A GiST opclass family extends this to stored regions without the B-tree rewrite, where a `float4` box key matched to `pgSphere`’s own representation beats every alternative tried, including `pgSphere`’s native GiST, on every corpus measured (Sect. 6.6).

It also closes two gaps neither established extension closes alone: ADQL’s own vocabulary with an operator form a planner can answer from an index, and stored regions that read and write as IVOA STC-S text with no translation step; because `skyregion` is tagged by kind rather than a family of types, one column holds circles and polygons together, which `pgSphere` cannot. Combined with one compact index serving both point and region queries, this makes `skycell` a strong candidate for a TAP service’s geometry layer (Sect. 6.4).

The advantage is a property of the workload, and scale sharpens it: raising the catalogue to 50 million rows widens the lead for regions of $30'$ and above (to a factor 3.2 at 1° cold) while cones of an arcminute and below stay level, since both touch a handful of pages regardless of table size and what is left is planning – also where a small relation, or a TAP server dominated by its own Python, favours `pgSphere` or leaves the choice immaterial. Choosing an index carefully matters when the scan dominates;

we have tried to say precisely which queries those are.

Data availability

The extension, its test suites, the benchmark harness and the raw results of every measurement quoted here are available at <https://github.com/jesusjuansalgado/skycell> under the MIT licence. Both corpora are generated from seeds by the scripts recorded there, reproducible without transferring data; `bench/run.sh` and scripts 07–10 reproduce every table, and the TAP A/B of Sect. 6.4 is recorded there too.

Acknowledgements. This work has made use of data from the European Space Agency (ESA) mission *Gaia* (<https://www.cosmos.esa.int/gaia>), processed by the *Gaia* Data Processing and Analysis Consortium (DPAC, <https://www.cosmos.esa.int/web/gaia/dpac/consortium>), funded by national institutions participating in the *Gaia* Multilateral Agreement. Only per-cell source counts were used, to build the density field of Sect. 6.1.

The implementation, benchmark harness and analysis were developed with the assistance of Anthropic’s Claude Code. Because that bears on how the results should be read: the geometric predicates and covering are checked by the brute-force suites of Sect. 4 against independent computations over millions of adversarially placed points rather than expected outputs; every indexed answer in the regression suites must equal a sequential scan with the exact predicate; and the error of Sect. 2.2 was found by those tests rather than by inspection, the argument for relying on them. The benchmark harness was itself audited by reproducing its headline comparison under a different protocol (Sect. 6.4), which is how the difference between the two came to light.

References

- Chilingarian, I., Bartunov, O., Richter, J., & Sigaev, T. 2004, in ASP Conf. Ser. 314, *Astronomical Data Analysis Software and Systems XIII*, ed. F. Ochsenbein, M. G. Allen, & D. Egret (San Francisco: ASP), 225
- Delli Veneri, M., & Salgado, J. 2026, *egernia*: an IVOA TAP 1.1 server for the SKA Observatory, Zenodo, doi:10.5281/zenodo.22845690
- Demleitner, M., Neves, M. C., Rothmaier, F., & Wambsganss, J. 2014, *Astron. Comput.*, 7–8, 27
- Dowler, P., Rixon, G., Tody, D., & Demleitner, M. 2019, *IVOA Recommendation: Table Access Protocol Version 1.1*
- ESA 1997, *The Hipparcos and Tycho Catalogues*, ESA SP-1200

- Fernique, P., Boch, T., Donaldson, T., et al. 2014, IVOA Recommendation: MOC – HEALPix Multi-Order Coverage map Version 1.0
- Gaia Collaboration, Prusti, T., de Bruijne, J. H. J., et al. 2016, *A&A*, 595, A1
- Górski, K. M., Hivon, E., Banday, A. J., et al. 2005, *ApJ*, 622, 759
- Gray, J., Nieto-Santisteban, M. A., & Szalay, A. S. 2006, The Zones Algorithm for Finding Points-Near-a-Point or Cross-Matching Spatial Datasets, Microsoft Research Technical Report MSR-TR-2006-52
- Koposov, S., & Bartunov, O. 2006, in ASP Conf. Ser. 351, *Astronomical Data Analysis Software and Systems XV*, ed. C. Gabriel, C. Arviset, D. Ponz, & S. Enrique (San Francisco: ASP), 735
- Kunszt, P. Z., Szalay, A. S., & Thakar, A. R. 2001, in *Mining the Sky*, ed. A. J. Banday, S. Zaroubi, & M. Bartelmann (Berlin: Springer), 631
- Moon, B., Jagadish, H. V., Faloutsos, C., & Saltz, J. H. 2001, *IEEE Trans. Knowl. Data Eng.*, 13, 124
- Ortiz, I., Lusted, J., Dowler, P., et al. 2008, IVOA Recommendation: Astronomical Data Query Language Version 2.0
- Rots, A. H. 2007, IVOA Recommendation: Space-Time Coordinate Metadata for the Virtual Observatory Version 1.33
- Salgado, J., González-Núñez, J., Gutiérrez-Sánchez, R., et al. 2017, *Astron. Comput.*, 21, 22
- Salgado, J., et al. 2020, in ASP Conf. Ser. 522, *Astronomical Data Analysis Software and Systems XXVII*, ed. P. Ballester, J. Ibsen, M. Solar, & K. Shortridge (San Francisco: ASP), 307